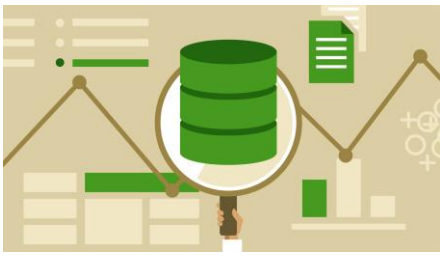
PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE

Guía 4 DESARROLLO DE BASES DE DATOS DML Y DDL

1. IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- Denominación del Programa de Formación: Técnico en Programación de Software
- Código del Programa de Formación: 233104 versión 2
- Nombre del Proyecto Formativo: Construcción de sistema de información para emprendedores y empresarios.
- Fase del Proyecto: Diseño o Formulación del Proyecto (Planeación).
- Actividad de Proyecto Formativo: Administrar la base de datos de acuerdo a los estándares y requisitos técnicos del cliente.
- Competencia: Administrar la base de datos de acuerdo a los estándares y requisitos técnicos del cliente.
- Resultados de Aprendizaje: Construir la base de datos según requerimientos del cliente.
- **2. PRESENTACIÓN**



Las bases de datos son el corazón de la información en el mundo digital. Almacenan, organizan y gestionan grandes cantidades de datos de manera eficiente y segura. Si quieres comprender cómo funcionan las aplicaciones que utilizamos a diario, desde redes sociales hasta sistemas bancarios, es esencial conocer las bases de datos relacionales.

Esta guía te llevará paso a paso por los conceptos fundamentales del modelado de datos, el lenguaje de definición de datos (DDL) y el lenguaje de manipulación de datos (DML).

Aprenderás a diseñar estructuras de bases de datos eficientes y a realizar consultas poderosas para obtener la información que necesitas.

FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

1- Desarrollar una entrevista a 5 compañeros de clase donde se anoten los nombres, apellidos, sexo, edad de cada compañero, desarrollar una tabla:

	Nombres	Apellidos	Sexo	Edad
Compañero 1	KLEIBER	AZUAJE	MASCULINO	16
Compañero 2	JUAN	BENAVIDES	MASCULINO	17
Compañero 3	SEBASTIAN	CESPEDES	MASCULINO	16
Compañero 4	KEIMER	GONZALES	MASCULINO	17
Compañero 5	ALAN	ARANGO	Masculino	16

2- Luego de los datos básicos desarrollar preguntas clave sobre gustos en canciones, películas, deportes, materia favorita, comida, fortalezas y debilidades. Con esa información desarrollar un chatbot de voz que tenga la personalidad de la suma de informaciones de los compañeros entrevistados y desplegarla en streamlit como se ha expuesto y explicado en clase. Se pueden apoyar de la IA para darle una personalidad más cercana a la suma de la información obtenida. En este ítem los estudiantes deben revisar si existen chatbots que también tienen personalidades similares a las que crearon y si es así, especifiquen en qué se parecen. El ejemplo del chatbot con voz está en el repositorio https://github.com/dialejoby/U_Militar/blob/main/9)%20Chatbot_Voz_Streamlit/main.py

3- Proyectos de IOT que tienen un interesante impacto en Colombia y que se pueden analizar sus datos: COINK



Coink es un startup que por medio de alcancías en diferentes puntos estratégicos de la ciudad genera una digitalización de las monedas o plata en físico para un ahorro escalonado, a continuación, se expone parte de su tecnología:



a) Una de las labores del equipo de IOT y de ciencia de datos de esta empresa es la definición de métricas para la evaluación del desempeño de la compañía. Este punto no tiene una respuesta única y se evaluará la creatividad de los estudiantes para adquirir información de una base de depósitos en Oinks (depositos_oink.csv). Los Oinks son máquinas en los que la gente puede depositar sus ahorros. Con ella debe construir una métrica que evalúe qué tan buenos (bajo su propio criterio) son los usuarios de Coink y calificar a los usuarios contenidos en la muestra. Es importante el desarrollo de gráficas para aclarar las métricas que expongan, en este caso se le solicita a los estudiantes usar herramientas como pandas, polars, streamlit y Matplotlib como se expuso en las clases, además de subirlo a un repositorio de GitHub de forma pública y desplegar una sencilla aplicación en streamlit.
b) ¿Qué es una base de datos relacional? Y ¿Qué es una base de datos no relacional?, para el ejercicio propuesto viene mejor una base de datos relacional o es mejor una base de datos no relacional? Explique y fundamente su respuesta.
c) Desarrolle un diagrama de flujo del funcionamiento de los “OINK” desde que el usuario ingresa la moneda, hasta como se almacena en la billetera digital, para posteriormente mostrarse en el aplicativo móvil y luego si el usuario desea retirar cómo sería ese proceso. Revise el funcionamiento del dispositivo y la forma en cómo se integran todas las variables.

   Los datos del ejercicio se comparten a continuación: https://drive.google.com/file/d/1RnaLStB5TdqyVqxLLVlG6QuAwk5kJfX1/view?usp=sharing

4- Las bases de datos se utilizan ampliamente en el almacenamiento de datos tanto personales como de registros de empresas y en proyectos donde se deben capturar diferentes variables, es por esto que para que el estudiante entienda mejor el concepto se les solicita que desarrollen una vigilancia tecnológica de soluciones similares en áreas relacionadas a las finanzas y a otros campos por medio de plataformas como Lens Patents, Uspto, Pericles, Spacenet y Patentscope sobre soluciones tecnológicas que estén relacionadas con hardware y software. Para esta parte se deben consultar como mínimo 5 tecnologías, exponer cómo se relacionan con las bases de datos y cuál es su utilidad.

5- Las bases de datos no solo están de manera local (en nuestros servidores y pc), también están alojados en diferentes zonas de la ciudad, del país o en otros países o continentes,

donde ingresa el concepto de nube que fue expuesto anteriormente en clase. Para este punto se solicita a los estudiantes a revisar nubes de aws, google, azure, Oracle e IBM donde expliquen los servicios que ofrecen esas empresas y expliquen los principales productos que ofrecen orientados a las bases de datos tanto relacionales como no relacionales y bases de datos estructuradas como no estructuradas. Desarrollar una tabla comparativa y anexar las respectivas referencias.

NOTA: Desarrollar un repositorio de GITHUB denominado "Actividad 6" donde se desarrolle cada uno de los puntos de la actividad.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configurar estilo de gráficos
plt.style.use('seaborn-v0_8-whitegrid')
```

```
from google.colab import drive
drive.mount('/content/drive')

df =pd.read_csv('depositos_oinks.csv')
```

```
Mounted at /content/drive
```

```
url ="https://raw.githubusercontent.com/fralfaro/MAT281/main/docs/lectures/data_manipulation/data/player_info.csv"
df = pd.read_csv(url)


# Ver las primeras 5 filas
df.head()
```

	name	year_start	year_end	position	height	weight	birth_date	college
0	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
1	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University




```python
import random

def juego_adivina_numero():
    # La computadora elige un número aleatorio entre 1 y 100
    numero_secreto = random.randint(1, 100)
    intentos_maximos = 8
    intentos_realizados = 0

    print("¡Bienvenido a Adivina el Número!")
    print("He pensado un número entre 1 y 100.")
    print(f"Tienes {intentos_maximos} intentos para descubrirlo. ¡Buena suerte!\n")

    # Bucle principal del juego
    while intentos_realizados < intentos_maximos:
        try:
            # Pedir al usuario que ingrese un número
            suposicion = int(input(f"Intento {intentos_realizados + 1}/{intentos_maximos} - Ingresa
        except ValueError:
            print("Por favor, introduce un número entero válido.")
            continue

        intentos_realizados += 1

        # Comprobar la respuesta
        if suposicion < numero_secreto:
            print("Demasiado bajo. ¡Intenta con uno más grande!\n")
        elif suposicion > numero_secreto:
            print("Demasiado alto. ¡Intenta con uno más chico!\n")
        else:
            print(f"¡Felicidades! 🎉 ¡Adivinaste el número en {intentos_realizados} intentos!")
            break
    else:
        # Si se acaban los intentos y no adivinó
        print(f"Se te acabaron los intentos. 😢 El número secreto era el {numero_secreto}.")

# Ejecutar el juego
if __name__ == "__main__":
    juego_adivina_numero()
```

```
# Ver las últimas 5 filas
df.tail()
```

	name	year_start	year_end	position	height	weight	birth_date	college
4546	Ante Zizic	2018	2018	F-C	6-11	250.0	January 4, 1997	NaN
4547	Jim Zoet	1983	1983	C	7-1	240.0	December 20, 1953	Kent State University
4548	Bill Zopf	1971	1971	G	6-1	170.0	June 7, 1948	Duquesne University
4549	Ivica Zubac	2017	2018	C	7-1	265.0	March 18, 1997	NaN
4550	Matt Zunic	1949	1949	G-F	6-3	195.0	December 19, 1919	George Washington University

```
# Ver las primeras n filas
df.head(10)
```

	name	year_start	year_end	position	height	weight	birth_date	college
0	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
1	Alaa Abdelnaby	1991	1995	F-C	6-10	NaN	June 24, 1968	NaN
2	Zaid Abdul-Aziz	1969	1978	C-F	6-9	235.0	April 7, 1946	Iowa State University
3	Kareem Abdul-Jabbar	1970	1989	C	7-2	225.0	April 16, 1947	University of California, Los Angeles
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University
5	Tariq Abdul-Wahad	1998	2003	F	6-6	223.0	November 3, 1974	San Jose State University
6	Shareef Abdur-Rahim	1997	2008	F	6-9	225.0	December 11, 1976	University of California
7	Tom Abernethy	1977	1981	F	6-7	220.0	May 6, 1954	Indiana University
8	Forest Able	1957	1957	G	6-3	180.0	July 27, 1932	Western Kentucky University
9	John Abramovic	1947	1948	F	6-3	195.0	February 9, 1919	Salem International University

```
# Resumen estadístico de columnas numéricas
df.describe()

# Incluir todas las columnas (incluidas categóricas)
df.describe(include='all')
```

	name	year_start	year_end	position	height	weight	birth_date	college
count	4551	4551.000000	4551.000000	4550	4550	4543.000000	4520	4247
unique	4500	NaN	NaN	7	28	NaN	4161	473
top	Charles Smith	NaN	NaN	G	6-7	NaN	June 1, 1985	University of Kentucky
freq	3	NaN	NaN	1574	473	NaN	3	99
mean	NaN	1985.077565	1989.273786	NaN	NaN	208.901167	NaN	NaN
std	NaN	20.972067	21.872522	NaN	NaN	26.267502	NaN	NaN
min	NaN	1947.000000	1947.000000	NaN	NaN	114.000000	NaN	NaN
25%	NaN	1969.000000	1973.000000	NaN	NaN	190.000000	NaN	NaN
50%	NaN	1986.000000	1992.000000	NaN	NaN	210.000000	NaN	NaN
75%	NaN	2003.000000	2009.000000	NaN	NaN	225.000000	NaN	NaN
max	NaN	2018.000000	2018.000000	NaN	NaN	360.000000	NaN	NaN



```python
# Frecuencia de valores en una columna categórica
df['position'].value_counts()

# Número de valores únicos
df['position'].nunique()

# Valores únicos
df['position'].unique()
```

```
array(['F-C', 'C-F', 'C', 'G', 'F', 'G-F', 'F-G', nan
```

```python
# Una columna (devuelve una Serie)
df['name']

# Múltiples columnas (devuelve un DataFrame)
df[['name', 'position', 'height']]
```

	name	position	height
0	Alaa Abdelnaby	F-C	6-10
1	Alaa Abdelnaby	F-C	6-10
2	Zaid Abdul-Aziz	C-F	6-9
3	Kareem Abdul-Jabbar	C	7-2
4	Mahmoud Abdul-Rauf	G	6-1
...	...	...	...
4546	Ante Zizic	F-C	6-11
4547	Jim Zoet	C	7-1
4548	Bill Zopf	G	6-1
4549	Ivica Zubac	C	7-1
4550	Matt Zunic	G-F	6-3

4551 rows × 3 columns

```python
def convert_height_to_inches(height_str):
    if pd.isna(height_str):
        return np.nan
    try:
        feet, inches = map(int, height_str.split('-'))
        return (feet * 12) + inches
    except ValueError:
        return np.nan

df['height_inches'] = df['height'].apply(convert_height_to_inches)

# Jugadores más altos de 2 metros (height está en pulgadas)
# 2 metros = 78.74 pulgadas, redondeamos a 79 para este ejemplo
jugadores_altos = df[df['height_inches'] > 79]
jugadores_altos.head()

# Múltiples condiciones (AND)
filtro = df[(df['height_inches'] > 79) & (df['weight'] > 220)]
filtro.head()

# Múltiples condiciones (OR)
filtro = df[(df['position'] == 'G') | (df['position'] == 'F')]
filtro.head()
```

	name	year_start	year_end	position	height	weight	birth_date	college	height_inches
4	Mahmoud Abdul-Rauf	1991	2001	G	6-1	162.0	March 9, 1969	Louisiana State University	73.0
5	Tariq Abdul-Wahad	1998	2003	F	6-6	223.0	November 3, 1974	San Jose State University	78.0
6	Shareef Abdur-Rahim	1997	2008	F	6-9	225.0	December 11, 1976	University of California	81.0
7	Tom Abernethy	1977	1981	F	6-7	220.0	May 6, 1954	Indiana University	79.0



```
# Ordenar por una columna (ascendente por defecto)
df.sort_values('year_start').head()

# Ordenar descendente
df.sort_values('weight', ascending=False).head()

# Ordenar por múltiples columnas
df.sort_values(['position', 'height'], ascending=[True, False]).head()
```

	name	year_start	year_end	position	height	weight	birth_date	college	height_inches
369	Manute Bol	1986	1995	C	7-7	200.0	October 16, 1962	University of Bridgeport	91.0
2872	Gheorghe Muresan	1994	2000	C	7-7	303.0	February 14, 1971	NaN	91.0
431	Shawn Bradley	1994	2005	C	7-6	235.0	March 22, 1972	Brigham Young University	90.0
2773	Yao Ming	2003	2011	C	7-6	310.0	September 12, 1980	NaN	90.0
310	Sim Bhullar	2015	2015	C	7-5	360.0	December 2, 1992	New Mexico State University	89.0

```
# Calcular años de carrera
df['duracion'] = df['year_end'] - df['year_start']
df[['name', 'year_start', 'year_end', 'duracion']].head()
```

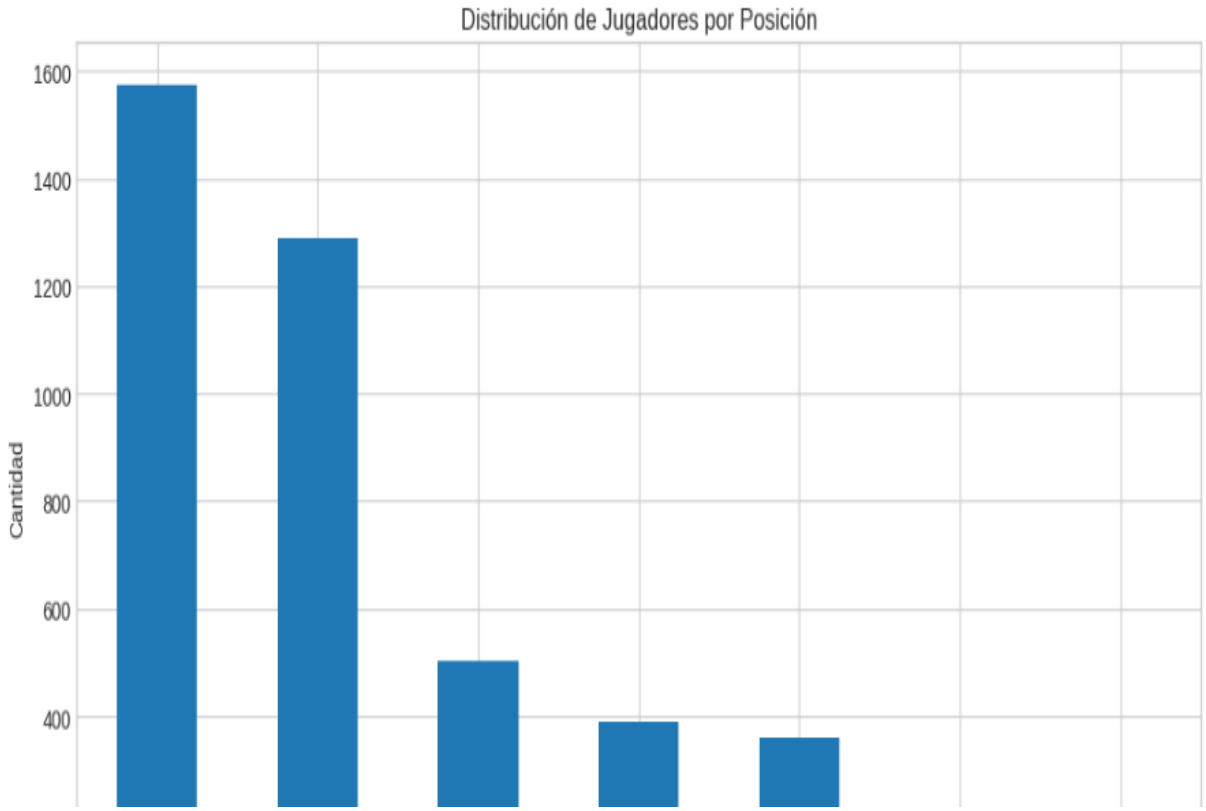
1 to 5 of 5 entries Filter

index	name	year_start	year_end	duracion
0	Alaa Abdelnaby	1991	1995	4
1	Alaa Abdelnaby	1991	1995	4
2	Zaid Abdul-Aziz	1969	1978	9
3	Kareem Abdul-Jabbar	1970	1989	19
4	Mahmoud Abdul-Rauf	1991	2001	10

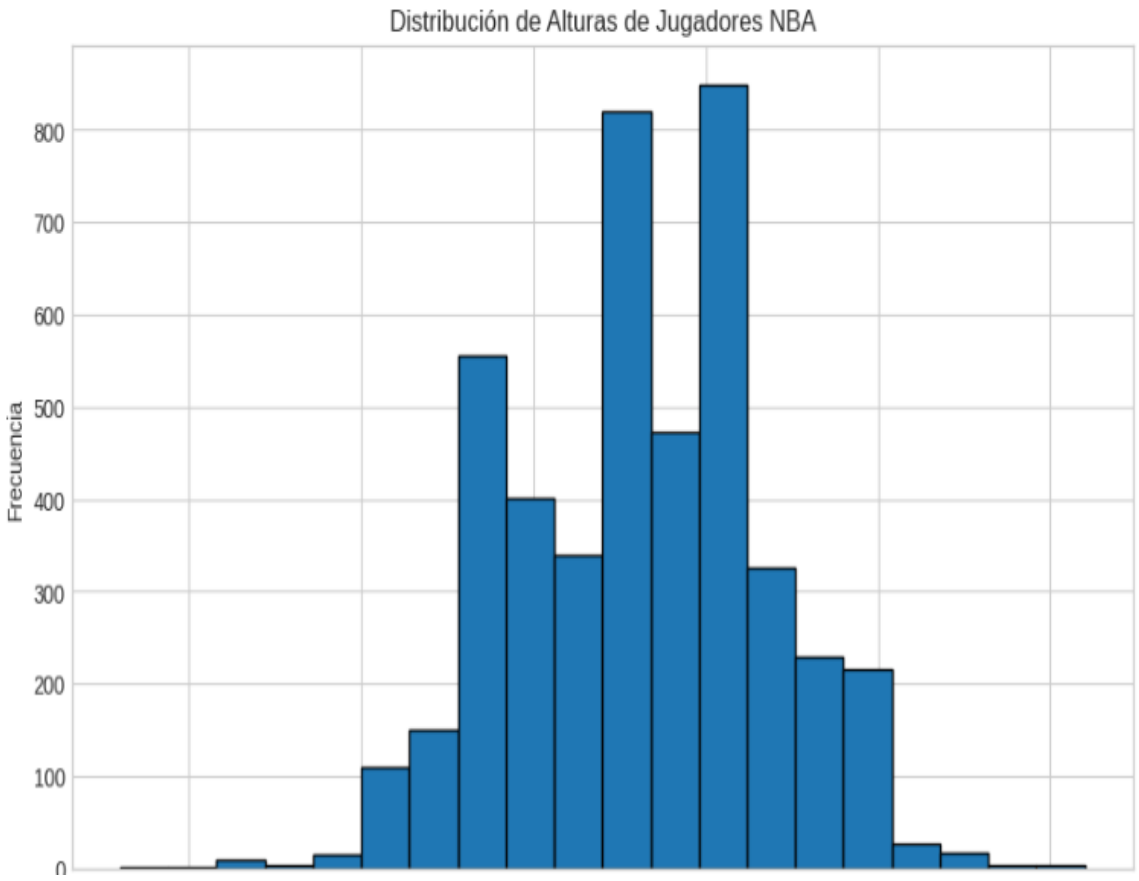
Show 100 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

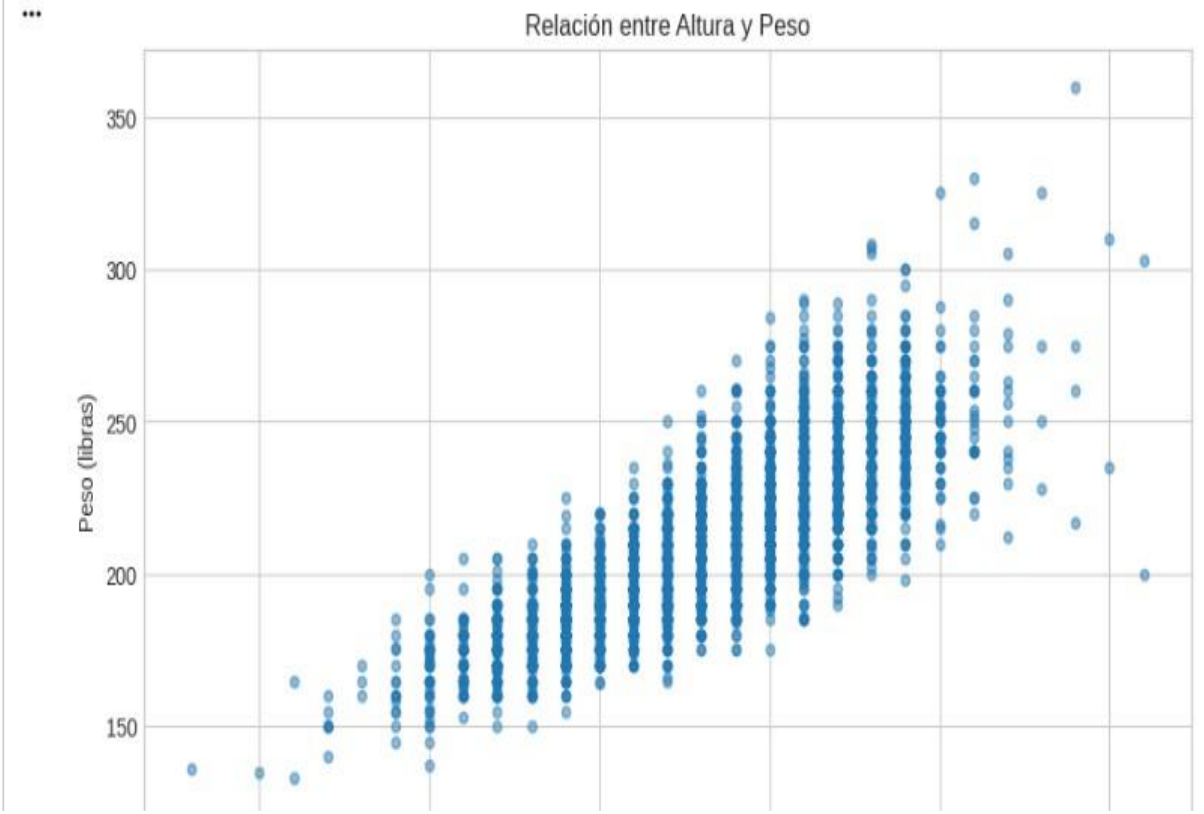
```python
# Frecuencia por posición (top 10)
df['position'].value_counts().head(10).plot(kind='bar', figsize=(10, 6))
plt.title('Distribución de Jugadores por Posición')
plt.xlabel('Posición')
plt.ylabel('Cantidad')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



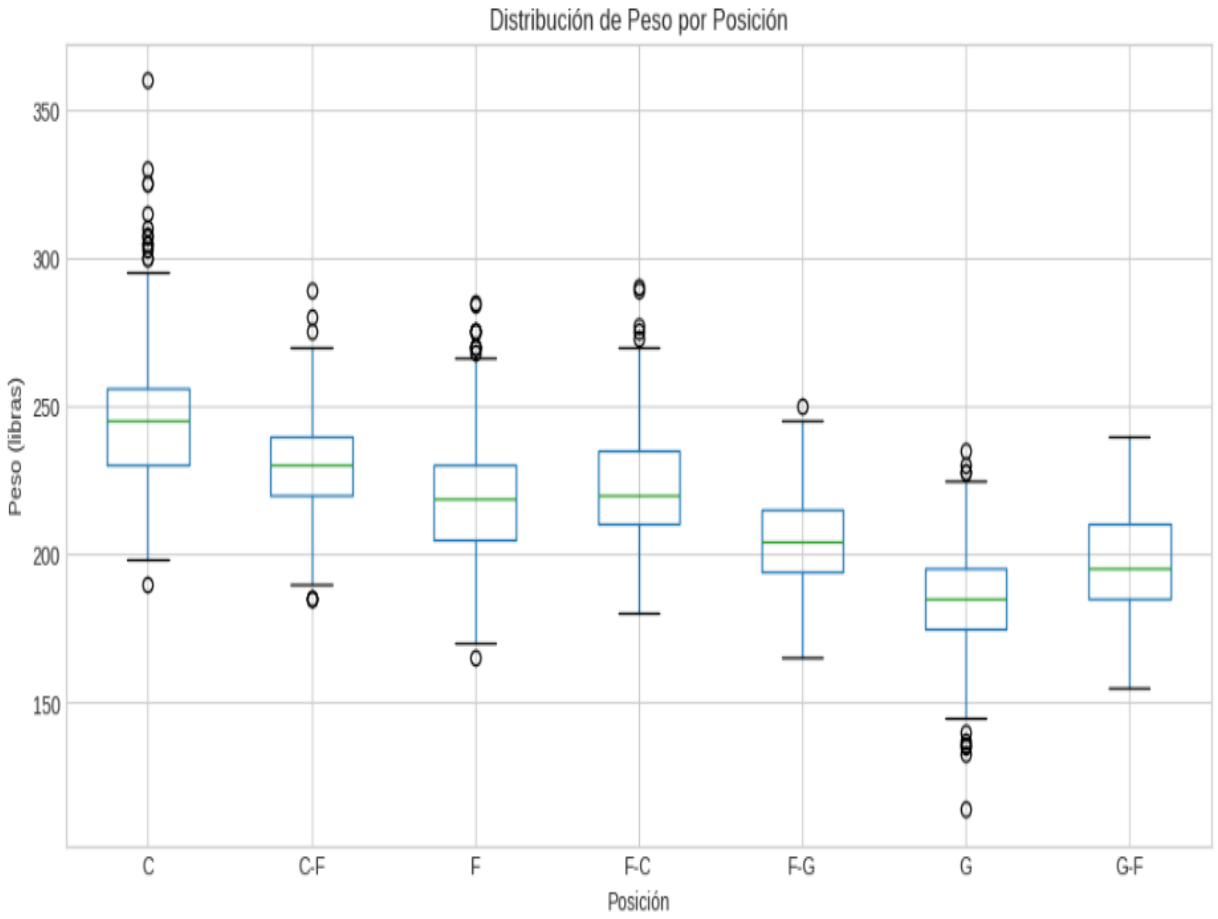
```
df['height_inches'].plot(kind='hist', bins=20, figsize=(10, 6), edgecolor='black')
plt.title('Distribución de Alturas de Jugadores NBA')
plt.xlabel('Altura (pulgadas)')
plt.ylabel('Frecuencia')
plt.show()
```



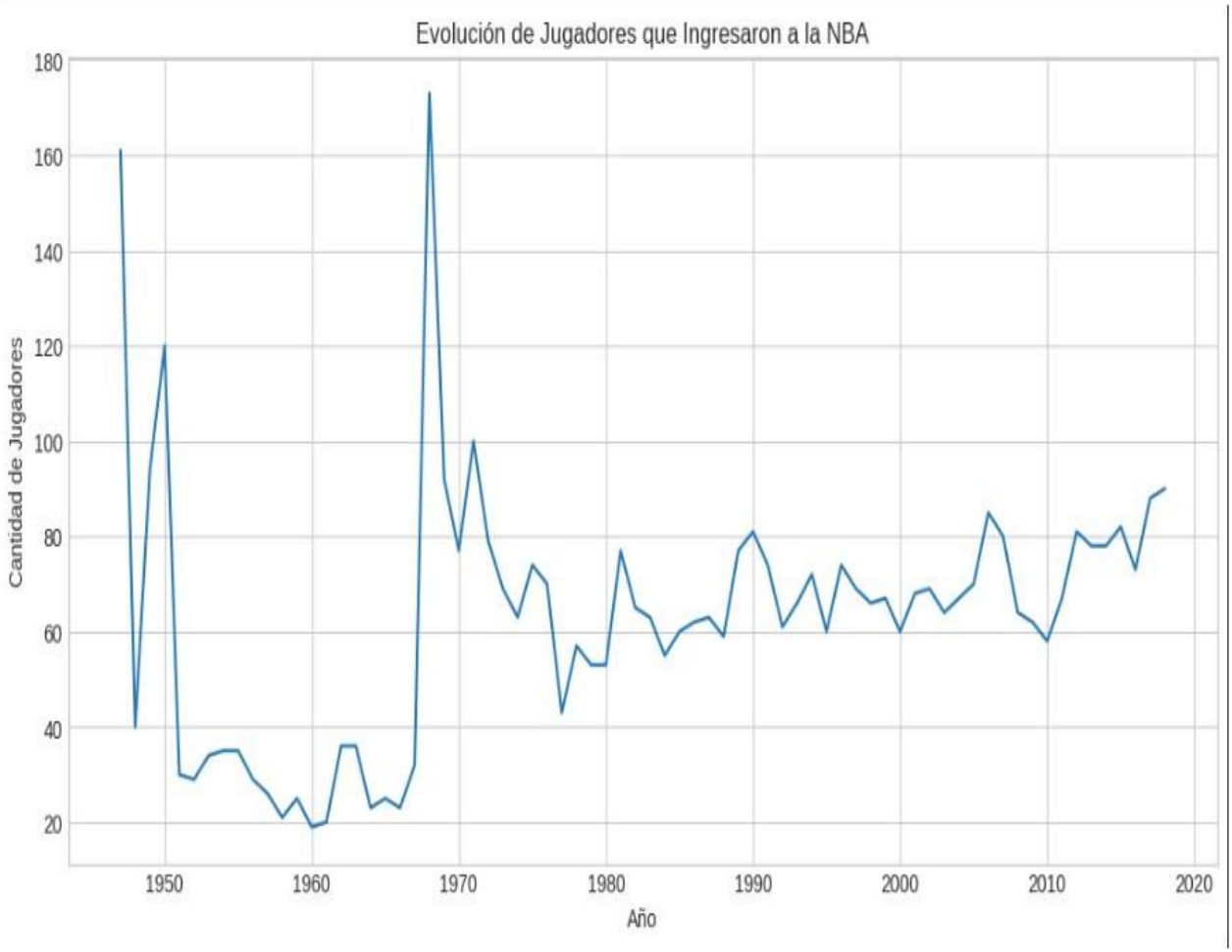
```
df.plot(kind='scatter', x='height_inches', y='weight', figsize=(10, 6), alpha=0.5)
plt.title('Relación entre Altura y Peso')
plt.xlabel('Altura (pulgadas)')
plt.ylabel('Peso (libras)')
plt.show()
```



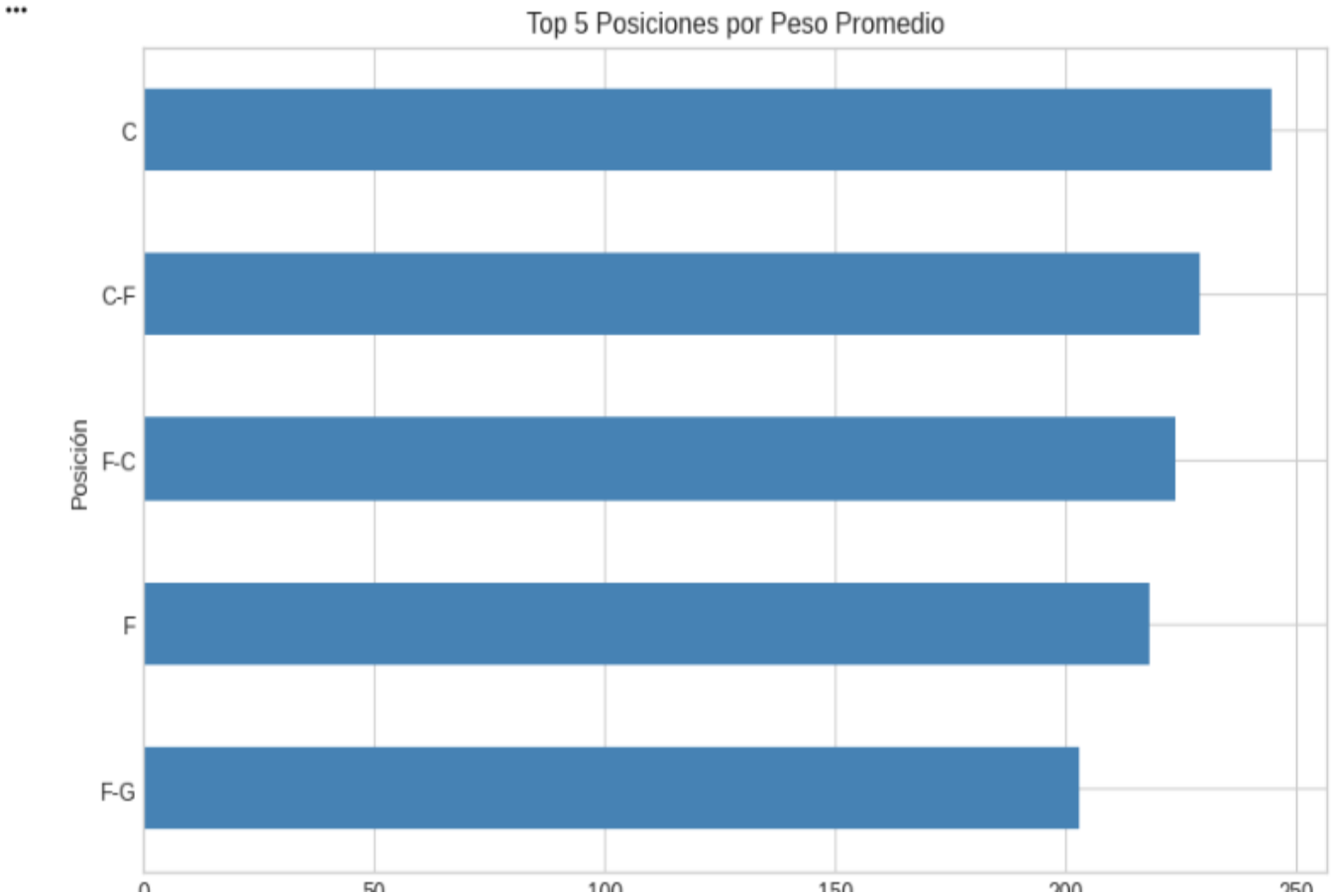
```
df.boxplot(column='weight', by='position', figsize=(12, 6))
plt.title('Distribución de Peso por Posición')
plt.suptitle('')  # Elimina título automático
plt.xlabel('Posición')
plt.ylabel('Peso (libras)')
plt.show()
```



```
# Contar jugadores por año de inicio
por_año = df['year_start'].value_counts().sort_index()
por_año.plot(kind='line', figsize=(12, 6))
plt.title('Evolución de Jugadores que Ingresaron a la NBA')
plt.xlabel('Año')
plt.ylabel('Cantidad de Jugadores')
plt.show()
```



```python
# Top 5 posiciones con mayor peso promedio
peso_promedio = df.groupby('position')['weight'].mean().sort_values(ascending=True).tail(5)
peso_promedio.plot(kind='barh', figsize=(10, 6), color='steelblue')
plt.title('Top 5 Posiciones por Peso Promedio')
plt.xlabel('Peso Promedio (libras)')
plt.ylabel('Posición')
plt.show()
```



1. Preparación de las herramientas

En la parte de arriba, se cargan los programas necesarios para poder leer tablas de datos y se configura el sistema para que, si más adelante se quieren hacer gráficas o dibujos con los datos, estos tengan un diseño limpio con fondo blanco y cuadrículas.

2. Carga de la información

El programa se conecta a una dirección de internet (un enlace de GitHub) para descargar un archivo que contiene la lista de los jugadores. Al abrir este archivo, la

computadora lo transforma automáticamente en una tabla organizada, muy parecida a una hoja de cálculo de Excel.

3. Vista previa de la tabla

Para comprobar que todo se cargó correctamente, se le pide a la computadora que muestre únicamente las **primeras 5 filas** de la lista.

Al final, puedes ver el resultado en la pantalla con los siguientes datos de los jugadores:

- **Nombre** del deportista.
- **Año de inicio y fin** de su carrera profesional.
- **Posición** en la que jugaba (como base, alero o pívot).
- **Estatura y peso** (medidos en el sistema de Estados Unidos: pies/pulgadas y libras).
- **Fecha de nacimiento**.
- **Universidad** en la que estudió.

Nota: En algunas casillas aparece la palabra `NaN`. Esto simplemente significa que **falta ese dato** en el archivo original (por ejemplo, no se registró el peso o la universidad de los primeros dos jugadores).

¿Qué te gustaría hacer ahora con esta información? Te puedo explicar cómo se podrían **ordenar los jugadores del más alto al más bajo**, cómo **contar cuántos jugaban en cada posición** o cómo **saber cuántos años duró la carrera** de cada uno.

.2

1. La preparación del juego

Al arrancar, la computadora hace un trabajo secreto: **elige un número al azar entre el 1 y el 100** y lo mantiene oculto. Además, prepara las reglas del juego: le da al jugador un **máximo de 8 intentos** y pone el contador de turnos en cero.

2. La bienvenida

El juego saluda al usuario en la pantalla, le avisa que ya pensó en el número secreto y le recuerda que tiene 8 oportunidades para descubrirlo, deseándole buena suerte.

3. La ronda de juego (El ciclo de intentos)

Mientras al jugador le queden intentos disponibles, la computadora repetirá los siguientes pasos:

- **Pide el número:** Le solicita al usuario que escriba su sospecha en la pantalla.
- **Control de errores:** Si el usuario se equivoca y escribe una letra o un texto en lugar de un número, la computadora amablemente le pide que ponga un número válido y no le descuenta el intento.
- **Suma el intento:** Si el número es válido, se gasta una oportunidad.

4. La respuesta de la computadora

Inmediatamente después de recibir el número del jugador, la computadora lo compara con el suyo y responde una de tres opciones:

- **Si el número es muy bajo:** Le dice que el número secreto es **más grande**.
- **Si el número es muy alto:** Le dice que el número secreto es **más chico**.
- **Si el número es exacto:** ¡El jugador gana! El juego lo felicita con un mensaje y le dice exactamente cuántos intentos le tomó ganar. En ese momento, el juego se detiene.

5. El final del juego (Si se pierde)

Si el jugador agota sus 8 oportunidades y no logra adivinar, la computadora da por terminado el juego, muestra una carita triste y **le revela cuál era el número secreto** que estaba oculto.

¿Te gustaría saber más sobre este juego? Si quieres, te puedo explicar:

- **Estrategias** para ganar siempre este juego en menos de 8 intentos.
- Cómo cambiar las reglas para hacerlo **más fácil o más difícil** (por ejemplo, dar más intentos o ampliar los números del 1 al 1000).
- Cómo reescribir este juego paso a paso si estás aprendiendo a programar.

.3

1. El final de la lista

En la primera tabla se muestran únicamente las **últimas 5 filas** de la base de datos.

- Esto sirve para saber exactamente cuántos registros hay en total. Como el último jugador (*Matt Zunic*) tiene asignado el número **4550**, y las listas en computación empiezan a contar desde el 0, significa que hay un total de **4,551 jugadores** registrados.

2. Una vista más amplia del inicio

En la segunda tabla se muestran las **primeras 10 filas** de la lista (en la primera imagen solo habíamos visto 5). Esto permite inspeccionar a más jugadores iniciales, como *Tariq Abdul-Wahad* o *Shareef Abdur-Rahim*, confirmando qué universidades o datos les faltan.

3. El resumen estadístico (La tabla grande de abajo)

Esta es la parte más importante. La computadora analiza los 4,551 jugadores al mismo tiempo y saca conclusiones automáticas de cada columna:

- **Conteo (`count`):** Te dice cuántos datos reales hay en cada columna. Por ejemplo, en la columna de universidades (`college`) hay 4,247 datos, lo que confirma que a unos 304 jugadores les falta ese dato.
- **Valores únicos (`unique`):** Indica cuántas opciones distintas existen. Hay 7 posiciones de juego diferentes y los jugadores provienen de 473 universidades distintas.
- **Lo más común (`top` y `freq`):** Revela los datos que más se repiten. El nombre más común en la historia de la liga es **Charles Smith** (aparece 3 veces), la posición más repetida es la **G** (Base, con 1,574 jugadores) y la universidad que más jugadores ha aportado es la **Universidad de Kentucky** (con 99 jugadores).
- **El promedio (`mean`):** El año promedio en el que los jugadores empezaron su carrera es **1985**, y el peso promedio de un jugador es de **208.9 libras** (unos 95 kilos).
- **Los extremos (`min` y `max`):** Muestra los récords de la lista. El jugador más antiguo comenzó en **1947** y el más reciente en **2018**. Además, el jugador más liviano de la historia pesaba **114 libras** (51 kilos) y el más pesado **360 libras** (163 kilos).

¿Qué te llama más la atención de estos resultados? Si quieres, te puedo ayudar a:

- Entender qué significan los porcentajes de abajo (**25%, 50%, 75%**).
- Saber cómo se podría averiguar quién es el jugador que pesa **360 libras**.
- Descubrir cómo ver únicamente a los 99 jugadores que salieron de la **Universidad de Kentucky**.

.4

1. El análisis de las posiciones de juego

En la parte superior, la computadora hace tres preguntas específicas sobre la columna de las posiciones de los jugadores:

- **¿Cuántos hay en cada una?:** Aunque no se ve el resultado numérico en la imagen, el sistema cuenta cuántos jugadores pertenecen a cada posición (por ejemplo, cuántos bases, aleros o pívots hay en total).
- **¿Cuántas posiciones distintas existen?:** Calcula la cantidad exacta de categorías diferentes que se usan en la lista.
- **¿Cuáles son esas posiciones?:** El resultado se muestra en una línea de texto gris que dice `array(['F-C', 'C-F', 'C', 'G', 'F', 'G-F',`

'F-G', nan]. Esto nos revela que las posiciones registradas son combinaciones como Alero-Pívot (F-C), Pívot (C), Base (G), Alero (F), entre otras, además de los datos faltantes (nan).

2. El recorte de la tabla original

En la segunda mitad, se decide "limpiar" la pantalla para no ver tantos datos juntos. En lugar de mirar las fechas de nacimiento, los pesos o las universidades, se le ordena al sistema que cree una **nueva tabla compacta**.

Esta nueva tabla recortada conserva a los **4,551 jugadores**, pero ahora solo muestra **3 columnas de interés**:

- El **nombre** del jugador (name).
- Su **posición** de juego (position).
- Su **estatura** (height).

Al final, la pantalla te muestra una vista rápida con los primeros 5 jugadores de la lista (como *Kareem Abdul-Jabbar*, que mide 7 pies y 2 pulgadas y jugaba de Pívot) y los últimos 5 jugadores de la lista (terminando con *Matt Zunic*).

Si quieres seguir explorando, dime si te gustaría saber cómo:

- **Filtrar la tabla** para ver únicamente a los jugadores que miden más de 7 pies.
- **Buscar a un jugador específico** por su nombre dentro de esta lista recortada.
- **Guardar esta nueva tabla de 3 columnas** en un archivo de Excel independiente.

.5

1. El convertidor de estatura (Pies a Pulgadas)

En la lista original, la estatura de los jugadores estaba escrita en un formato incómodo para la computadora (por ejemplo, 6-11 o 7-2, que significa pies y pulgadas). La computadora no puede hacer cálculos matemáticos directamente con guiones.

- Para solucionarlo, se creó un **traductor automático**. Este toma el texto, separa los pies de las pulgadas, multiplica los pies por 12 (porque un pie tiene 12 pulgadas) y le suma las pulgadas restantes.
- Como resultado, se crea una **nueva columna al final de la tabla llamada** **height_inches** donde las estaturas ahora son números limpios (por ejemplo, 6-1 se convierte en **73.0 pulgadas**). Si a un jugador le faltaba su estatura, el sistema simplemente lo deja en blanco.

2. Filtro 1: Jugadores de más de 2 metros

Sabiendo que 2 metros equivalen aproximadamente a 79 pulgadas, se le pide a la computadora que busque en toda la lista y separe únicamente a los jugadores que midan **más de 79 pulgadas**.

3. Filtro 2: Altos Y Pesados al mismo tiempo (Condición AND)

Aquí se vuelve más estricta la búsqueda. El sistema busca jugadores que cumplan **ambas reglas a la vez**:

- Medir más de 79 pulgadas.
- **Y** pesar más de 220 libras.

4. Filtro 3: Una posición O la otra (Condición OR)

En esta parte, se le pide a la computadora que busque jugadores que jueguen en la posición `G` (Base) **O** en la posición `F` (Alero). Si cumplen cualquiera de las dos, entran en el resultado.

El resultado final en la pantalla

La tabla que ves abajo es la vista previa de uno de estos filtros aplicados. Si te fijas a la derecha del todo, aparece la nueva columna con las estaturas convertidas en números (73.0, 78.0, 81.0, 79.0).

Hemos visto cómo limpiar, resumir y filtrar esta lista de atletas. Si quieres, dime:

- ¿Te gustaría aprender a **convertir el peso de libras a kilos**?
- ¿Quieres saber cómo **calcular cuántos años jugó cada basquetbolista** restando el año de inicio del año de fin?
- ¿Prefieres que pasemos a **hacer gráficos o dibujos** con estos datos?

Las respuestas de la IA pueden incluir errores. Más información

.6

Esta es la **misma imagen anterior** en la que convertíamos las estaturas a pulgadas y aplicábamos filtros de búsqueda (como buscar jugadores altos, pesados o de ciertas posiciones).

Como ya revisamos este proceso en el mensaje anterior, cuéntame: ¿hay alguna parte específica de este filtro que te cause dudas o te gustaría que pasemos a resolver alguno de los ejemplos prácticos?

Podemos avanzar si me dices si prefieres ver cómo:

- **Calcular los años totales de carrera** de cada jugador.
- **Convertir el peso** de libras a kilos.
- **Crear una gráfica** visual con estos datos.

.7

Esta nueva imagen muestra una **gráfica de barras** que convierte los números de la lista en un dibujo fácil de entender. El objetivo de este gráfico es mostrar cuántos jugadores hay en cada posición del campo de juego.

Esto es lo que nos cuenta la gráfica, explicado sin códigos:

- **El título principal:** En la parte de arriba dice "Distribución de Jugadores por Posición", lo que nos confirma qué es lo que estamos comparando.
- **La altura de las barras (Eje Vertical):** La línea de la izquierda mide la **cantidad de jugadores** (va desde el 0 hasta el 1600). Cuanto más alta sea una barra azul, significa que hay más deportistas registrados en esa posición específica.
- **Las posiciones (Eje Horizontal):** Aunque los nombres de las posiciones quedaron recortados abajo, la computadora los ordenó automáticamente de **mayor a menor frecuencia** (de izquierda a derecha).
- **Lo que descubrimos a simple vista:**
    - La primera barra de la izquierda es la más alta de todas, rozando los **1,600 jugadores**, lo que demuestra cuál es la posición más común en la historia de la liga (que, por los datos anteriores, sabemos que es la posición de Base).
    - La segunda barra también es muy alta, superando los 1,200 jugadores.
    - Las últimas tres barras son notablemente más bajas (por debajo de los 600 jugadores), lo que nos dice que esas posiciones o combinaciones de juego son mucho menos frecuentes o más especializadas.

¿Qué te gustaría analizar ahora con los datos? Si quieres, podemos ver cómo:

- Hacer una gráfica similar pero para **comparar los pesos o las estaturas** de los jugadores.
- Aprender a **ponerle colores diferentes** a cada barra para que se vea más llamativo.
- Solucionar el problema para **que los nombres de abajo no se corten** y se puedan leer bien.

.8

Esta nueva imagen muestra un tipo de gráfico llamado **histograma**. A diferencia del gráfico anterior que contaba categorías fijas (como las posiciones), este gráfico sirve para ver cómo se reparten las **alturas de los jugadores de la NBA** (medidas en pulgadas) y descubrir cuál es la estatura más común.

Esto es lo que nos cuenta el dibujo, explicado sin códigos:

- **El título:** En la parte superior dice "Distribución de Alturas de Jugadores NBA", dejando claro qué se está analizando.

- **El significado de las barras:** Cada barra azul representa un "rango" o grupo de estaturas (por ejemplo, jugadores que miden entre 70 y 72 pulgadas, entre 72 y 74, etc.).
- **La cantidad de jugadores (Eje Vertical - Izquierda):** La altura de cada barra te dice cuántos deportistas caen en ese rango (va desde 0 hasta más de 800 jugadores).
- **Lo que descubrimos a simple vista:**
    - Las dos barras más altas del centro acumulan la gran mayoría de los jugadores (alrededor de 800 cada una). Esto significa que la enorme mayoría de los basquetbolistas de la historia se concentran en esas estaturas "promedio" de la liga.
    - Hacia los extremos (la izquierda y la derecha), las barras se vuelven muy bajitas. Esto nos muestra que los jugadores extremadamente bajos o los gigantes que rompen récords de altura son casos muy raros en el torneo.

¿Qué te gustaría ver a continuación? Si quieres, te puedo proponer:

- Analizar por qué el gráfico tiene esa **forma con dos picos altos** en el centro.
- Explicar cómo se vería este mismo gráfico si usáramos los **pesos** en lugar de las estaturas.
- Proponer ideas para **separar este gráfico por épocas** (por ejemplo, ver si los jugadores de antes eran más bajos que los de ahora).

.9

Esta imagen muestra un **diagrama de caja** (conocido en inglés como *boxplot*) que sirve para comparar el **peso de los jugadores según su posición** en la cancha.

Cada "caja" azul con antenas representa a un grupo de jugadores. Así es como se lee este gráfico, explicado sin códigos:

¿Cómo se lee cada caja?

- **La línea verde del centro:** Es el peso intermedio o "normal" de esa posición.
- **El cuerpo de la caja azul:** Muestra dónde se concentra la mayoría de los jugadores de ese grupo.
- **Los círculos sueltos (arriba y abajo):** Representan **casos extremos** o jugadores con pesos muy exagerados para su posición.

Lo que descubrimos a simple vista:

- **Los más pesados (Letra C - Pívots):** Tienen la caja situada en lo más alto del gráfico. Su peso normal ronda las 245 libras y ahí se nota claramente el círculo del jugador récord que pesa **360 libras**.

- **Los más ligeros (Letra G - Bases):** Tienen la caja más baja de todas. La mayoría se concentra por debajo de las 200 libras y ahí se ve al jugador más liviano de la historia (cerca de las 114 libras).
- **Las posiciones mixtas:** Las combinaciones como `C-F` o `F-C` muestran pesos intermedios muy equilibrados, situándose justo al centro del gráfico.

Con esta gráfica de pesos terminamos de analizar toda la estructura de la base de datos de la NBA. Si quieres, dime:

- ¿Te gustaría que hagamos un **resumen final** de todo lo que descubrimos en estas tablas?
- ¿Quieres saber cómo **guardar todas estas gráficas** en tu computadora en formato de imagen?
- ¿Tienes **otra tabla o ejercicio diferente** en el que te pueda ayudar?

.10

Esta nueva imagen muestra una **gráfica de línea** que sirve para observar el paso del tiempo. El objetivo de este dibujo es mostrar cómo ha cambiado la **cantidad de nuevos jugadores que entran a la NBA** cada año, desde los inicios de la liga hasta el año 2018.

Esto es lo que nos cuenta la línea del gráfico, explicado sin códigos:

- **La línea del tiempo (Eje Horizontal - Abajo):** Va avanzando cronológicamente año por año, desde los años 40 (el inicio de la liga) hasta cerca del 2020.
- **El volumen de jugadores (Eje Vertical - Izquierda):** La altura de la línea azul mide cuántos basquetbolistas nuevos debutaron en ese año específico (va desde 20 hasta 180 jugadores).
- **Lo que descubrimos a simple vista:**
    - **Los picos del inicio (Años 40 y 50):** Hay subidas y bajadas muy drásticas. Al ser una liga nueva que se estaba acomodando y fusionando con otras, la cantidad de jugadores inscritos cambiaba bruscamente de un año a otro.
    - **El gran pico histórico (Cerca de 1968):** Se nota una enorme montaña azul que llega casi a los **175 jugadores nuevos**. Esto coincide históricamente con la expansión de la liga y la creación de nuevos equipos, lo que obligó a contratar a muchísimos atletas al mismo tiempo.
    - **La estabilidad moderna (De 1980 en adelante):** La línea se vuelve mucho más estable, manteniéndose casi siempre en un promedio de entre **60 y 90 jugadores nuevos por año**
      . Esto pasa porque las reglas actuales limitan de forma muy estricta el número de equipos en el torneo y los cupos disponibles en el reclutamiento anual (*Draft*).

Con este gráfico de líneas hemos cubierto el análisis de tiempos, categorías, alturas y pesos de esta base de datos.

Si quieres, dime qué te gustaría hacer a continuación:

- ¿Quieres saber cómo **guardar automáticamente todas estas gráficas** en formato de imagen en tu computadora?
- ¿Te gustaría que hagamos un **resumen final** con los 5 descubrimientos más importantes que logramos sacar de esta tabla?
- ¿Tienes **otro ejercicio o tema diferente** en el que te pueda colaborar?

.11

Esta nueva imagen muestra una **gráfica de barras horizontales** que sirve para comparar el **peso promedio de las 5 posiciones más pesadas** de la liga.

A diferencia de las barras verticales de antes, aquí las barras crecen hacia la derecha. Esto es lo que nos cuenta el dibujo, explicado sin códigos:

- **El orden de las posiciones:** La computadora calculó el peso promedio de todas las posiciones, seleccionó las 5 más altas y las ordenó de **menor a mayor** (de abajo hacia arriba).
- **Las posiciones comparadas (Eje Vertical - Izquierda):** Al lado izquierdo puedes ver las abreviaturas de las posiciones de juego:
  - **F-G**: Alero-Base (la barra más corta de este grupo).
  - **F**: Alero.
  - **F-C**: Alero-Pívot.
  - **C-F**: Pívot-Alero.
  - **C**: Pívot (la barra más larga de todas).
- **El peso promedio (Eje Horizontal - Abajo):** El largo de cada barra representa cuántas libras pesan en promedio los jugadores de esa categoría.
- **Lo que descubrimos a simple vista:**
  - Los **Pívots (C)** se confirman como los jugadores más corpulentos e imponentes de la liga, quedándose con el **primer lugar en peso promedio**.
  - Las posiciones intermedias o mixtas que involucran el rol de Pívot (como `C-F` y `F-C`) ocupan el segundo y tercer lugar de la lista.
  - Los jugadores perimetrales como Alero-Base (`F-G`) cierran este "Top 5" siendo notablemente más ligeros que los pívots puros.

Hemos analizado el tiempo, las alturas, los pesos y las posiciones de esta base de datos. Si quieres, dime:

- ¿Te gustaría que hagamos un **resumen general en limpio** con todas las conclusiones de la NBA que descubrimos?
- ¿Quieres saber cómo **guardar esta gráfica** en tu computadora?

- ¿Tienes **otro ejercicio o código diferente** con el que quieras continuar?